Problem Statement :

Implement FCFS CPU Scheduling for the following scenario where No of processes = 5 and Process informations are-

P id	AT	BT
P0	3	1
P1	5	3
P2	2	2
P3	1	2
P4	6	3

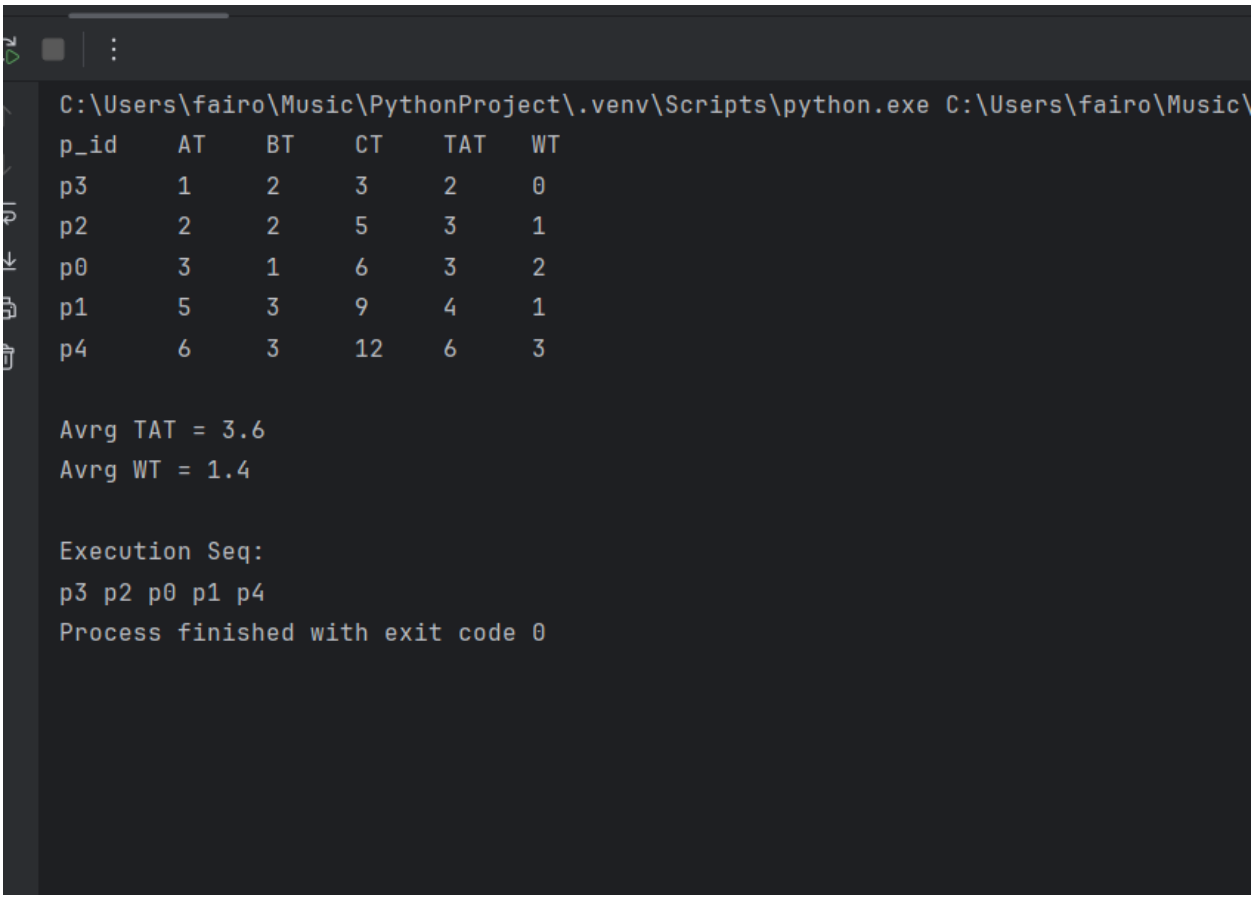
Calculate CT, TAT, WT, AVG TAT and AVG WT. Also represent the process execution sequence.

Source Code :

```
p_id = ["p0", "p1", "p2", "p3", "p4"]
at = [3, 5, 2, 1, 6]
bt = [1, 3, 2, 2, 3]

for i in range(len(at) - 1):
  for j in range(len(at) - 1 - i):
    if at[j] > at[j + 1]:
      at[j], at[j + 1] = at[j + 1], at[j]
      bt[j], bt[j + 1] = bt[j + 1], bt[j]
      p_id[j], p_id[j + 1] = p_id[j + 1], p_id[j]

ct = []
tat = []
wt = []
current_time = 0
sum_tat = 0
sum_wt = 0
```

```
for i in range(len(p_id)):

  if current_time < at[i]:
    current_time = at[i]

  current_time += bt[i]
  ct.append(current_time)

  tat.append(ct[i] - at[i])
  wt.append(tat[i] - bt[i])

  sum_tat += tat[i]
  sum_wt += wt[i]



print(f"{'p_id':<8}{'AT':<6}{'BT':<6}{'CT':<6}{'TAT':<6}{'WT':<6}")
for i in range(len(p_id)):
  print(
      f"{p_id[i]:<8}{at[i]:<6}{bt[i]:<6}{ct[i]:<6}{tat[i]:<6}{wt[i]:<6}"
  )

print("\nAvrg TAT =", sum_tat / len(p_id))
print("Avrg WT =", sum_wt / len(p_id))

print("\nExecution Seq:")
for i in range(len(p_id)):
  print(p_id[i], end=" ")
```

Output:



Discussion :

While implementing this program, I encountered several challenges and resolved them through trial, debugging, and logical analysis.

The first challenge was sorting the processes based on their Arrival Time. Initially, I considered using Python's built-in sorting method with a lambda expression. However, I was not familiar with how the lambda function works, so I found it difficult to implement correctly. To overcome this problem, I decided to use the **Bubble Sort** algorithm because it is simple, easy to understand, and I was more comfortable implementing it manually. While sorting the Arrival Time array, I also swapped the corresponding Burst Time and Process ID values to maintain the correct relationship among the process information.

Another issue I faced was related to variable naming and tracking the current execution time. Initially, I used t = 0 to represent the current execution time, but this caused

confusion while working with the program. I was having difficulty clearly identifying what the variable represented during the calculation. To solve this problem, I changed it to **current_time = 0**, which made the purpose of the variable much clearer. This helped me properly track the CPU's current execution time and successfully resolve the issue.

I also had to carefully implement the calculations of Completion Time (CT), Turnaround Time (TAT), and Waiting Time (WT). The program first checks whether the CPU is idle by comparing current_time with the Arrival Time. It then updates the current time and calculates the Completion Time based on the Burst Time. From CT, the TAT and WT are calculated for each process.

After debugging and testing the program several times, I was able to obtain the correct execution sequence of **p3 → p2 → p0 → p1 → p4**. The final output produced an average Turnaround Time of **3.6** and an average Waiting Time of **1.4**, indicating that the program successfully implemented the FCFS scheduling logic.

Conclusion:

This assignment provided me with practical experience in implementing the **First Come First Serve (FCFS) CPU Scheduling algorithm** using Python. During the implementation, I faced difficulties with sorting the processes, understanding the lambda expression, and properly tracking the current execution time. I solved these problems by using the **Bubble Sort** algorithm and replacing the unclear t variable with the more meaningful current_time variable.

The implementation also helped me understand how Arrival Time and Burst Time affect the execution order of processes in FCFS scheduling. After debugging and testing the program, I successfully obtained the execution sequence **p3 → p2 → p0 → p1 → p4**, with an average Turnaround Time of **3.6** and an average Waiting Time of **1.4**.

Overall, this assignment improved my understanding of FCFS scheduling as well as my programming, debugging, and problem-solving skills. It also helped me understand the importance of using clear variable names and choosing an algorithmic approach that is easier for me to understand and implement.